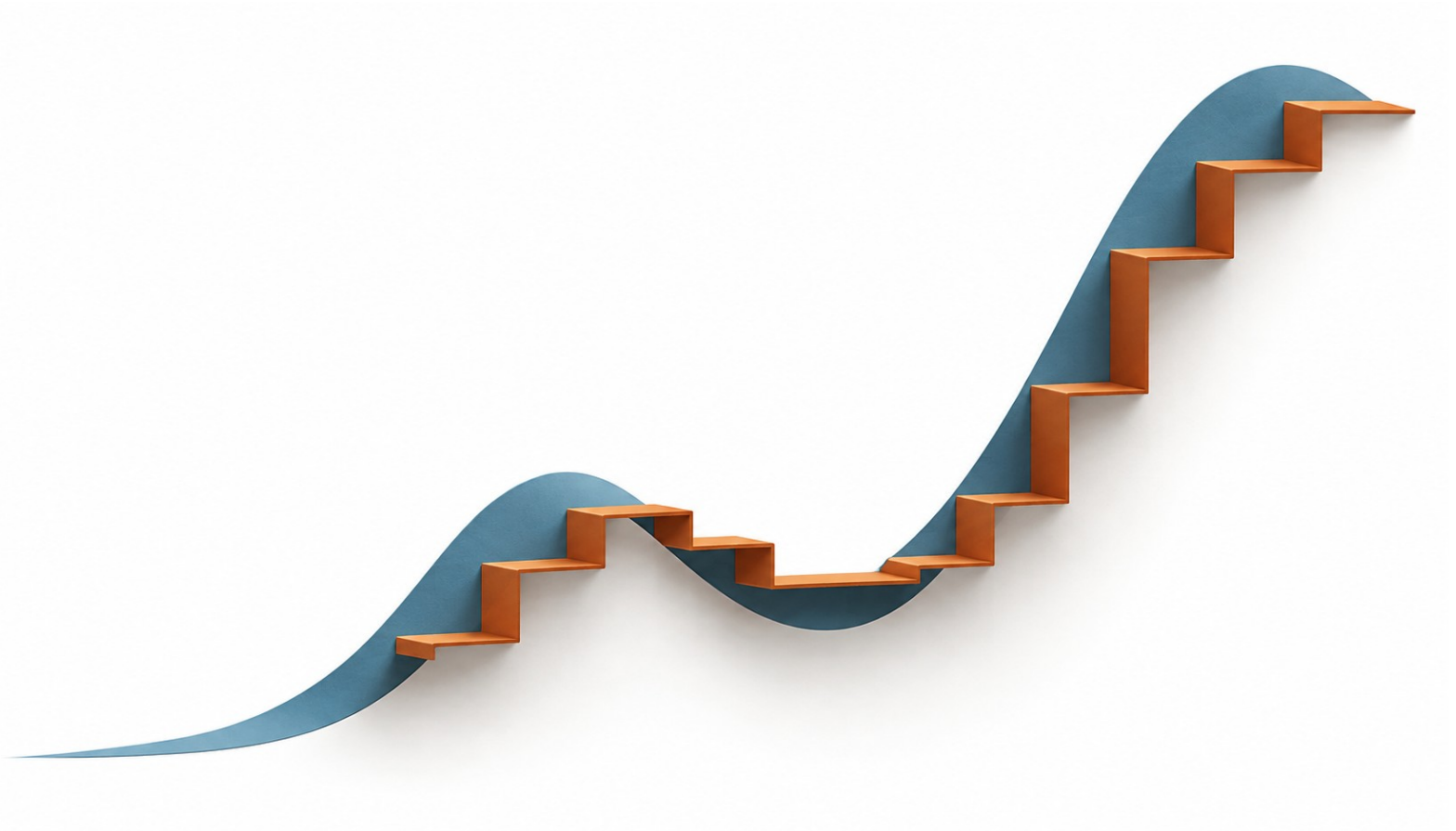


Numeric Representation and Quantization

THEORY SESSION 06B

The numbers inside the model

Foundations for understanding
reduced precision



24 September 2026 · Tiago Fonseca

The same network, another numeric representation

Architecture

The layers, connections and number of learned parameters can remain the same.

Numeric representation

The stored values use a different format. Their approximations can change the model's outputs.

Quantization can change precision without removing neurons or layers.

Bits and available codes

b bits provide 2^b distinct bit patterns.

2 bits

4 bits

8 bits

4 codes

16 codes

256 codes

The format defines what each pattern means.

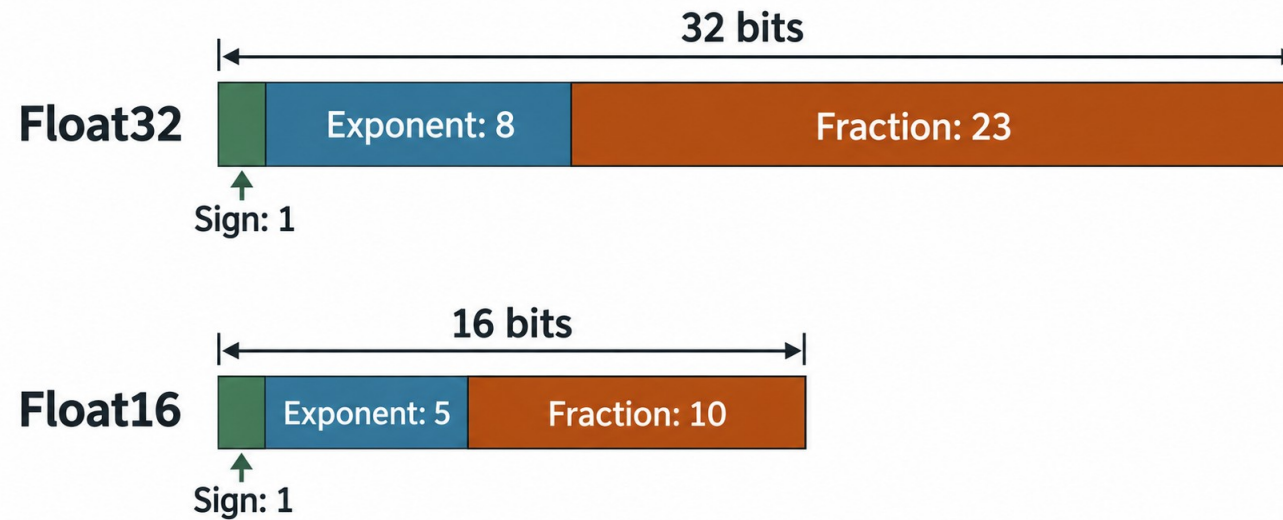
For uniform integer quantization, scale and zero-point give those codes their real values.

Float32, Float16 and INT8

Format	Bits / value	What it represents
Float32	32	Floating-point values
Float16	16	Floating-point values, with less precision and range
INT8	8	256 signed integer codes: –128 to 127

Floating point also has finite precision. Its spacing varies with magnitude.

Inside a floating-point number



The exponent controls magnitude. The fraction refines the value.

Float16 has fewer bits for both fields. Diagram widths are schematic.

Floating-point spacing changes with magnitude

Two pairs of adjacent Float16 values

Value	Next larger value	Gap
1	1.0009765625	0.0009765625
1024	1025	1

Larger magnitudes have coarser absolute spacing.

Examples of normal binary16 values. These are exact gaps.

Some decimal fractions need an approximation

0.5

Exactly representable in binary:

$$0.1_2 = 1/2$$

0.1

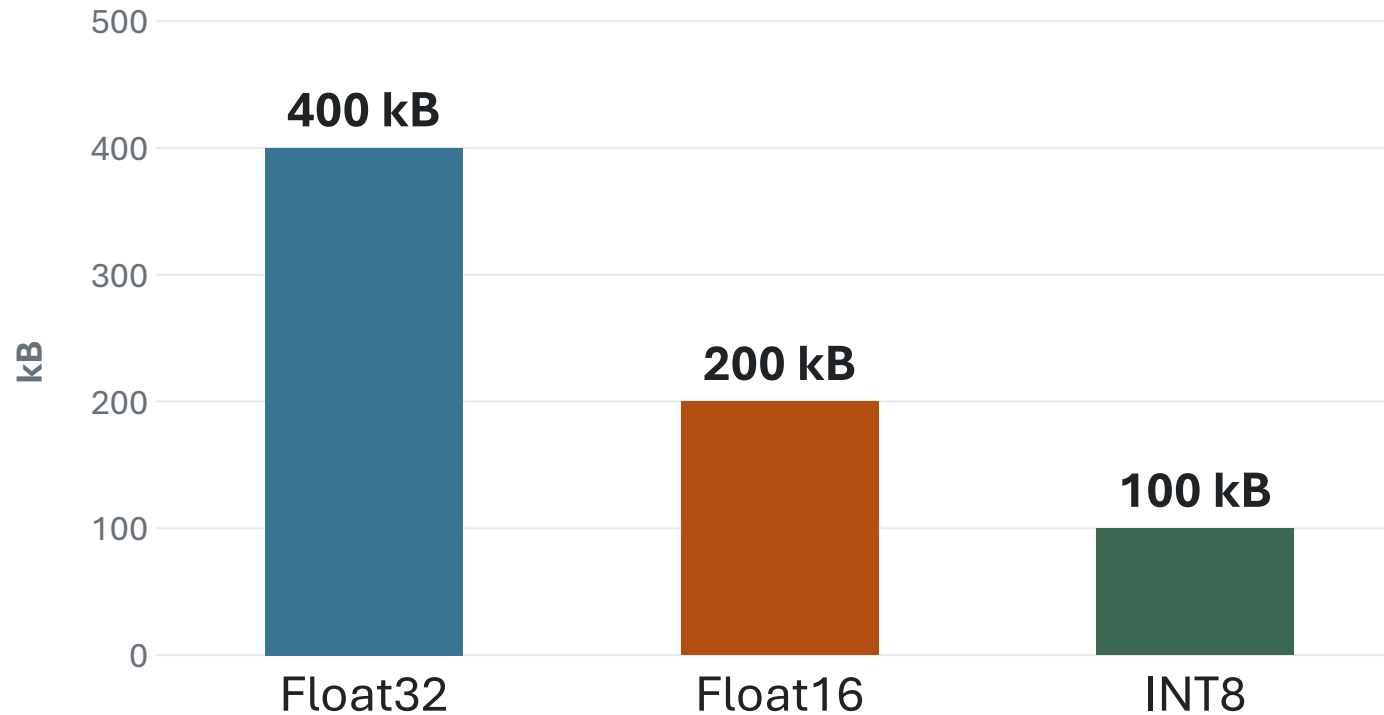
Its binary expansion repeats.

A finite format must round it.

Float32 stores decimal 0.1 as approximately 0.10000000149.

Finite precision is already present before INT8 quantization.

Raw storage for 100,000 weights



4 : 2 : 1

Bytes per stored weight

$100,000 \times \text{bits} / 8$

Theoretical weight data only. Excludes metadata, other tensors and runtime memory.

Decimal units: 1 kB = 1,000 bytes.

Storage precision and arithmetic precision

In the model file

Float16

Weights can occupy
16 bits per value.

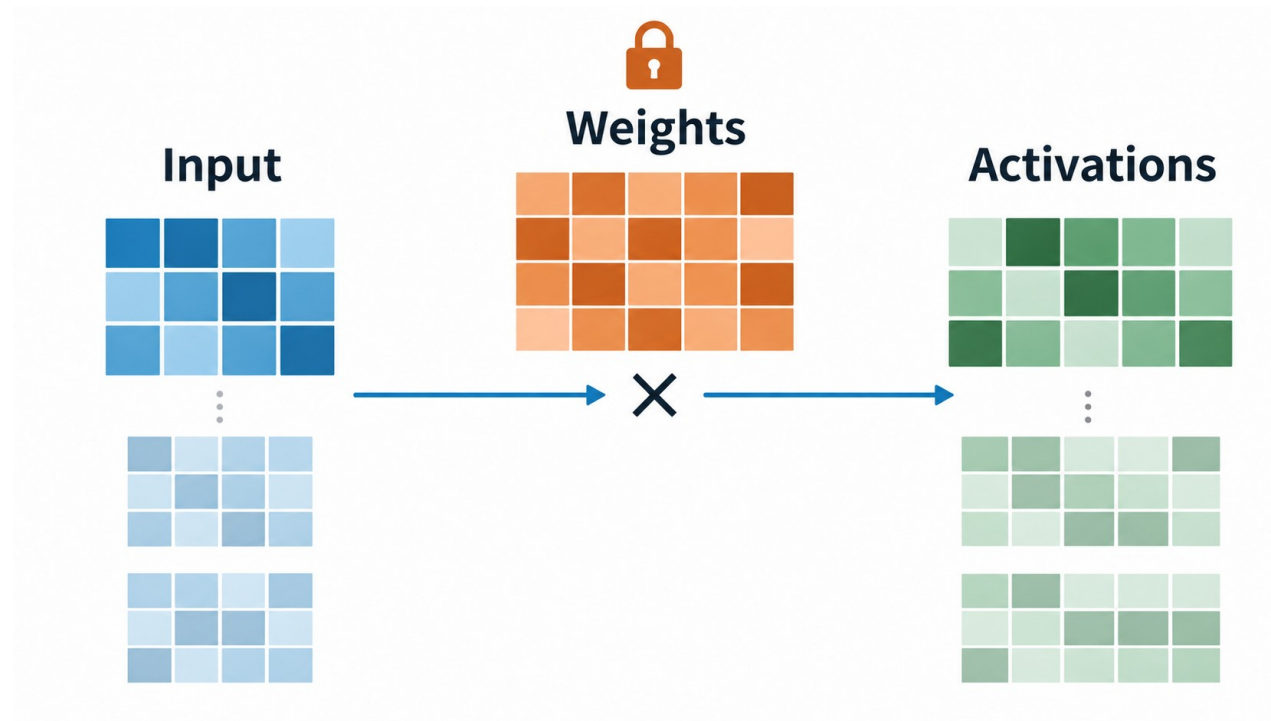
During execution

Float32

Some CPU paths expand
those weights for arithmetic.

The runtime and backend determine the arithmetic. Fewer stored bytes do not fix the speed.

Weights and activations



Weights

Learned parameters, normally fixed during inference.

Activations

Intermediate values that change with each input.

Range and resolution

Narrower range



Wider range



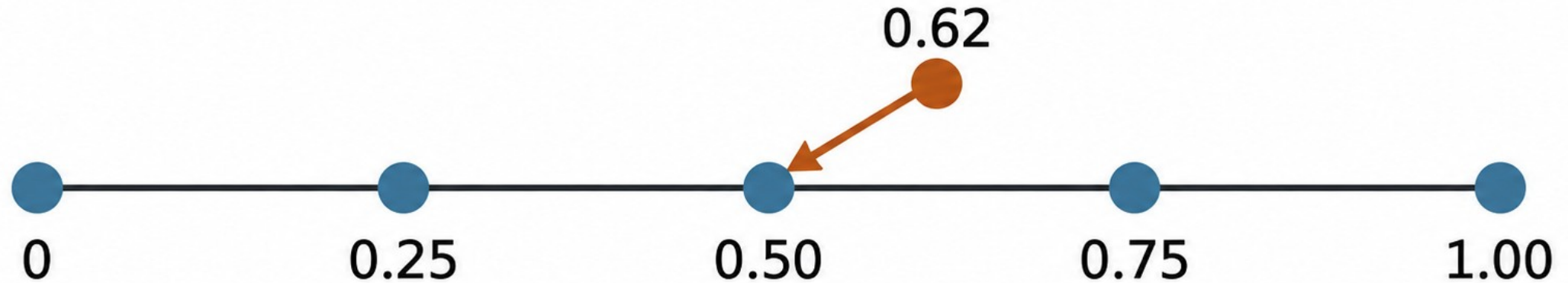
Same number of levels.

A wider range gives a larger step between adjacent values.

Illustrative uniform grids with five levels. These are not complete INT8 ranges.

A value moves to a representable level

Quantization approximates values using a finite set of levels.



Here, 0.62 becomes 0.50.

Illustrative grid with five levels and nearest-value rounding.

Scale and zero-point

$$x \approx s \times (q - z)$$

q Stored integer code

s Scale: the step size

z Code that represents zero

x Original real value

$s \times (q - z)$ reconstructs a real-valued approximation of x .

Encoding and reconstruction

Encode a real value as an integer code

$$q = \text{clip}(\text{round}(x / s) + z, q_{\min}, q_{\max})$$

Reconstruct its real-valued approximation

$$x' = s \times (q - z)$$

round selects the nearest integer. clip enforces the available code range.

One value through quantization

Original value

0.62

Integer code

$\text{round}(0.62 / 0.25) = 2$

Reconstructed value

$0.25 \times (2 - 0) = 0.50$

Scale = 0.25. Zero-point = 0. Absolute error = 0.12.

Zero-point preserves real zero

Scale = 0.5 Zero-point = 10

Integer code q	8	9	10	11	12
Real value	-1.0	-0.5	0.0	0.5	1.0

$q = 10$ represents $x = 0$.

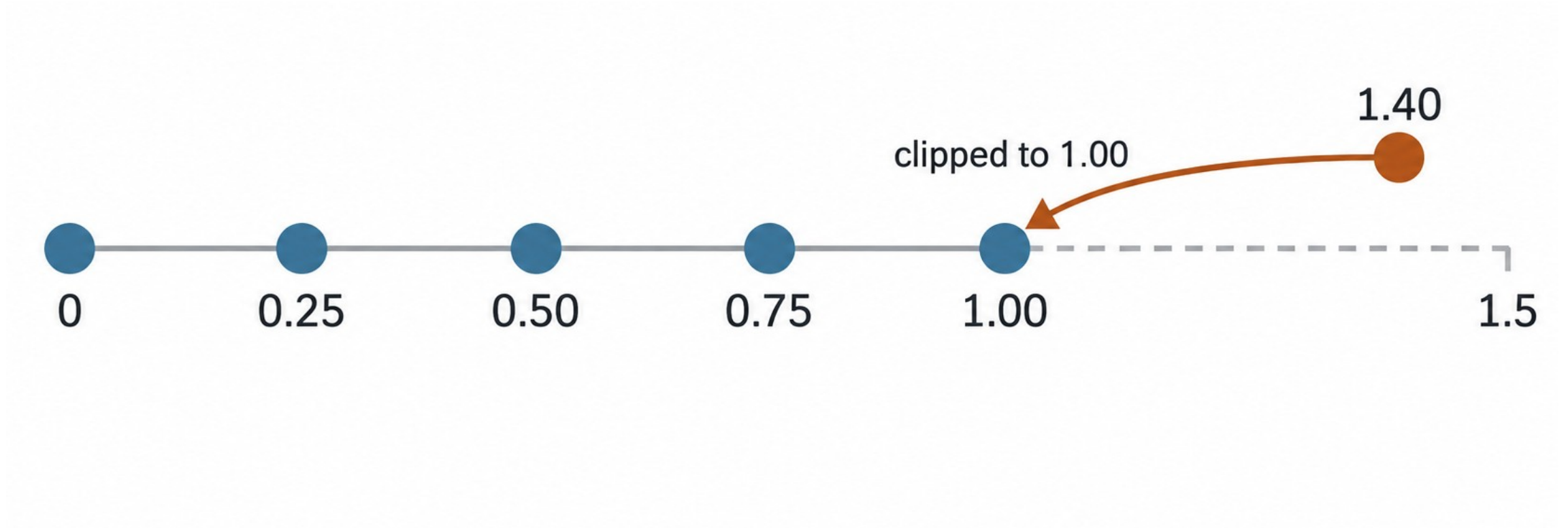
An integer code of zero does not always represent a real value of zero.

Signed codes and unsigned codes

The same real values can use different integer codes.

Real value	INT8 code	UINT8 code
-1.0	-10	118
0.0	0	128
1.0	10	138
Both: scale = 0.1	INT8: $z = 0$ Range: -128...127	UINT8: $z = 128$ Range: 0...255

Values outside the range are clipped



Rounding chooses a nearby level. Clipping stops at a limit.

Illustrative five-level quantizer limited to the interval $[0, 1]$.

The rounding error depends on the step

Nearest rounding, with no clipping

$$|x - x'| \leq s / 2$$

Scale 0.25

Maximum rounding error: 0.125

Our value 0.62

Reconstruction 0.50

Absolute error: 0.12

Clipping can produce a larger error.

Choosing a scale for an example range

256 levels give 255 intervals.

Example activation range: 0 to 5.10

$$s = 5.10 / 255 = 0.02$$

q = -128

represents 0.00

q = 127

represents 5.10

INT8 example with zero-point $z = -128$.

Calibration with representative data

Weights are already known.

Activation values depend on the inputs that pass through the model.

Calibration observes example inputs to estimate useful numeric ranges for those activations.

The examples should resemble the inputs the deployed model will receive.

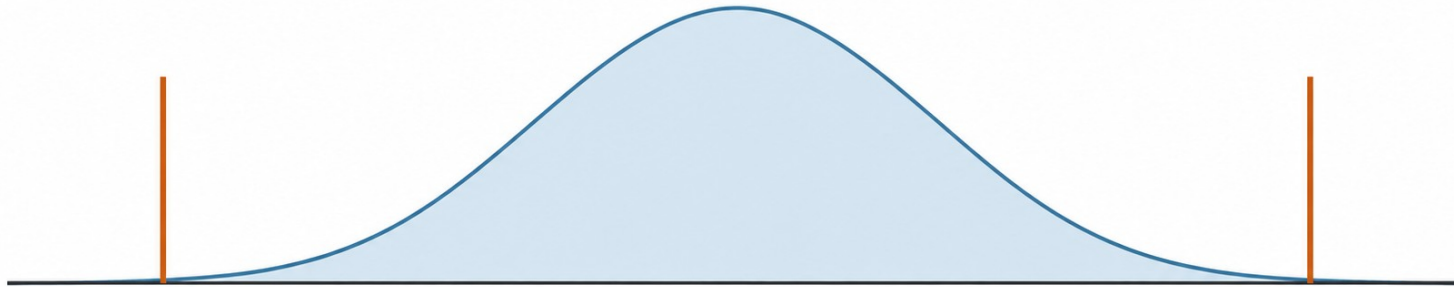
Training, calibration and testing

Stage	What it determines	What it uses
Training	Learned weights	Training examples and a learning objective
Calibration	Activation ranges for quantization	Representative input examples
Testing	Quality on unseen examples	A held-out evaluation set

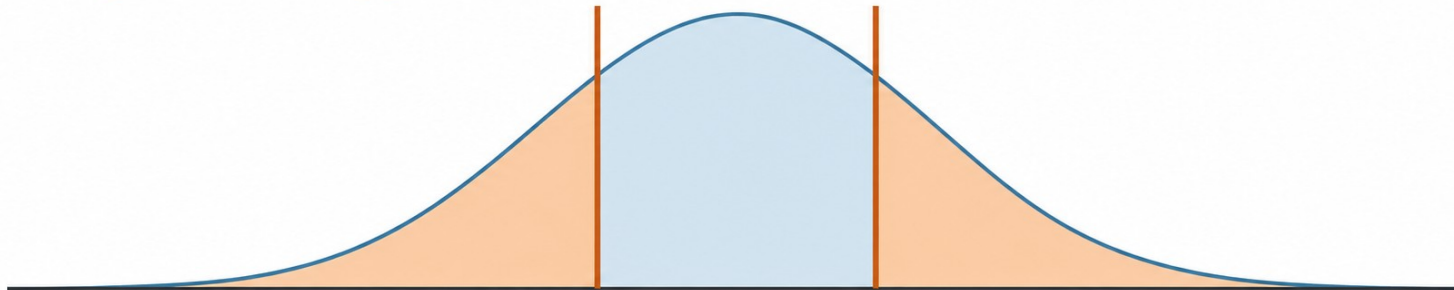
Calibration observes the model. It does not relearn the weights.

A calibration range can miss typical values

Range covers typical values



Range misses typical values



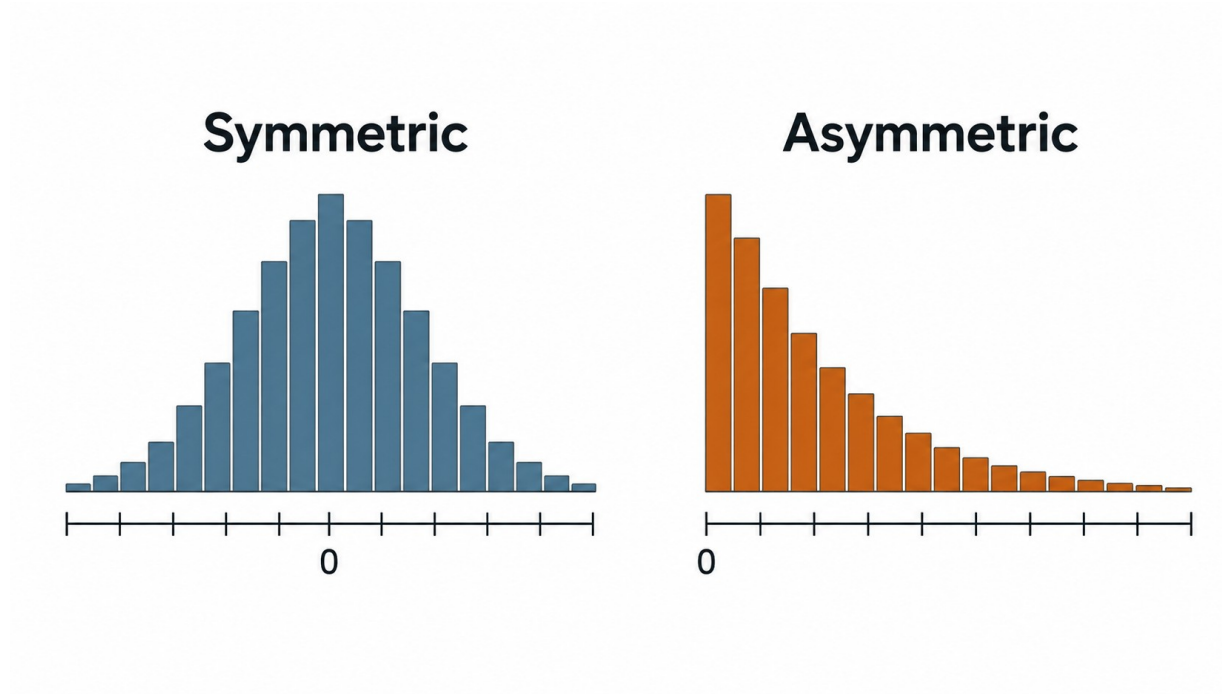
Coverage matters.

A range that is too narrow causes more clipping.

A very wide range gives coarser steps for the same number of levels.

Schematic distributions, not experimental results.

Symmetric and asymmetric quantization



Symmetric weights

Zero-point = 0

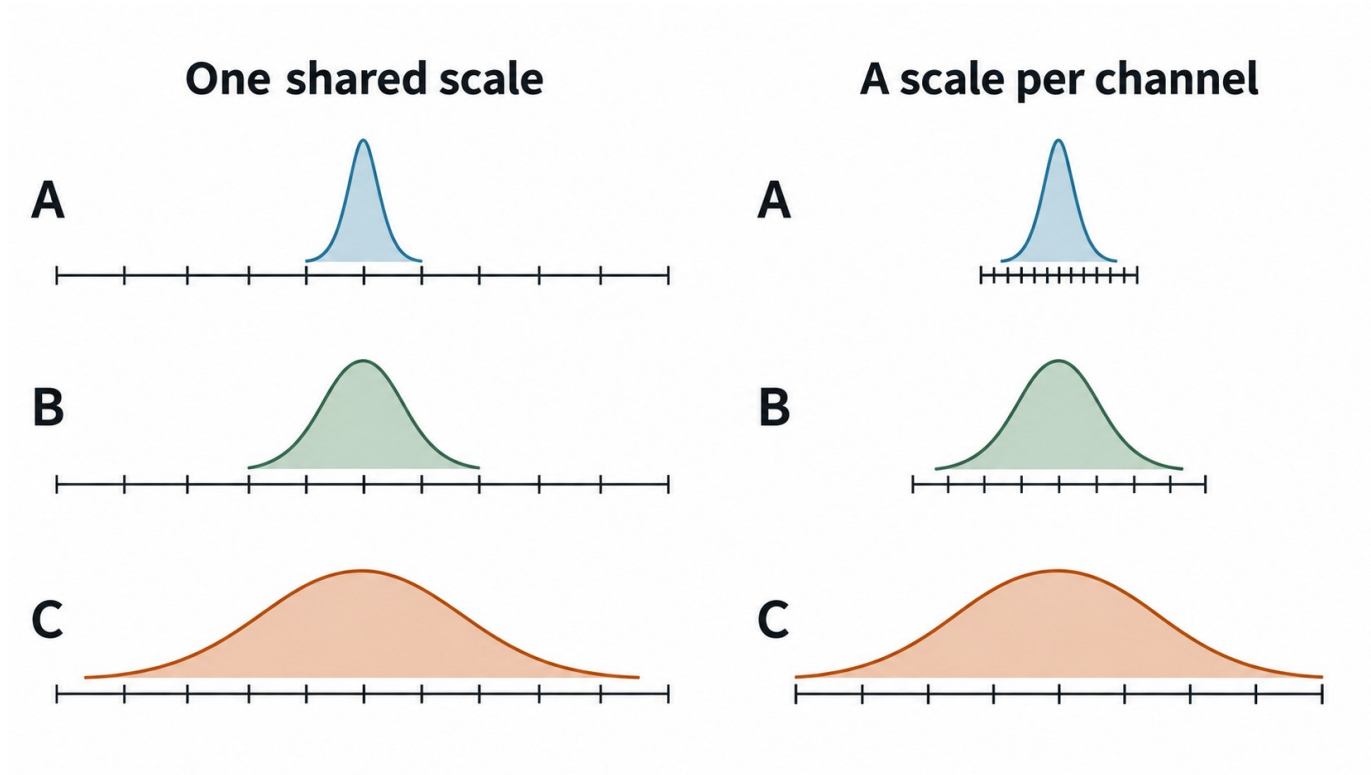
LiteRT INT8 codes: $-127 \dots 127$

Asymmetric activations

Zero-point can shift

LiteRT INT8 codes: $-128 \dots 127$

One scale for a tensor or one per channel



Channels with different ranges can benefit from different step sizes.

Per-channel quantization applies separate parameters along a selected tensor axis.

A small channel can lose useful detail

Weight ranges: channel A ± 0.1 , channel B ± 10

Quantization of A	Scale	Reconstruction of 0.02
Shared scale	$10 / 127 \approx 0.07874$	$q = 0$, value = 0
A's own scale	$0.1 / 127 \approx 0.0007874$	$q = 25$, value ≈ 0.0197

A finer local scale preserves this small value more closely.

Toy example. Symmetric codes $-127 \dots 127$, zero-point 0, nearest rounding.

Approximation errors can propagate

A simple operation: $y = w \times x$

Original weight

Approximate weight

$$0.62 \times 10 = 6.2$$

$$0.50 \times 10 = 5.0$$

Weight error: 0.12. Output error: 1.2.

Later operations can amplify or attenuate the effect.

INT8 operands need wider accumulation

One product

$$127 \times 127 = 16,129$$

A sum of 64 such products

$$64 \times 16,129 = 1,032,256$$

INT32 accumulation provides room for larger sums.

Requantization gives the output its own scale

Input scale 0.1 × weight scale 0.2 = product scale 0.02

Accumulator

Output tensor

$$30 \times 0.02 = 0.60$$

$$0.60 / 0.05 = 12$$

The same real result uses a different integer code at the output.

Toy example: zero-points 0, no bias, output scale 0.05, no clipping.

Post-training and quantization-aware training

After training

Post-training quantization

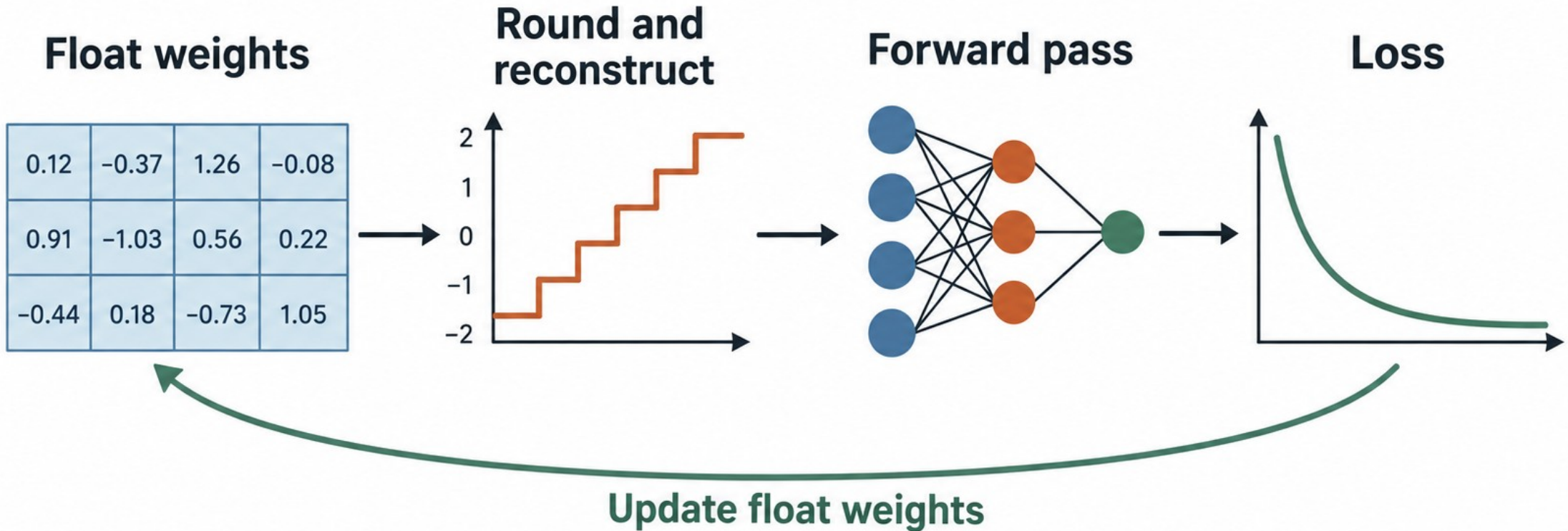
Quantize an existing trained model.
Calibration can estimate activation ranges.

During training

Quantization-aware training

Simulate quantization effects while learning, so the model can adapt its parameters.

What fake quantization does during training



The forward pass experiences rounding effects.

Training still maintains floating-point parameters. Conversion produces the deployed model.

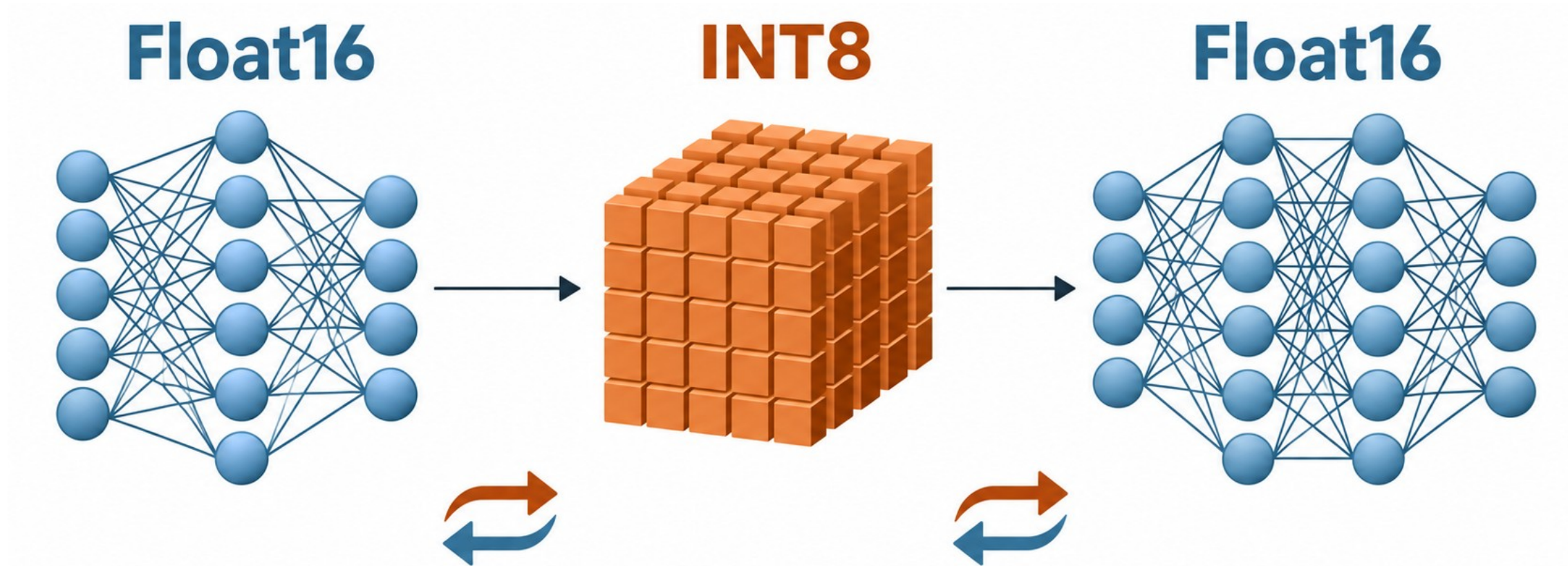
Common LiteRT post-training approaches

Approach	Stored weights	Activation treatment
Float16	Float16	Backend-dependent arithmetic
Dynamic range	INT8	Quantized at runtime in supported paths
Full integer	INT8	Ranges calibrated before deployment

Full integer quantization needs representative inputs.

Dynamic range can include float operations. Wider biases and accumulators still apply.

Mixed precision within a model



Different tensors or stages can use different numeric formats.

Illustrative arrangement. Useful choices depend on sensitivity and backend support.

A small change can alter the predicted class

Illustrative classifier scores

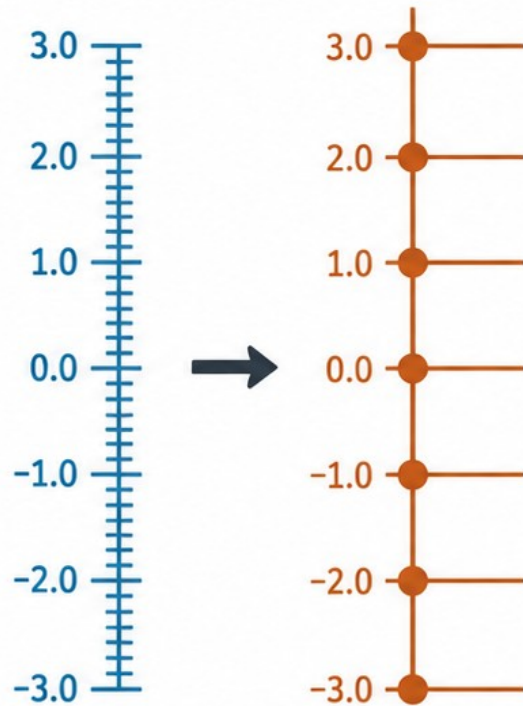
Class	Original model	Approximate model
A	0.51	0.49
B	0.49	0.51

The winning class changes when the order of the scores changes.

Invented scores for explanation. These are not measured quantization results.

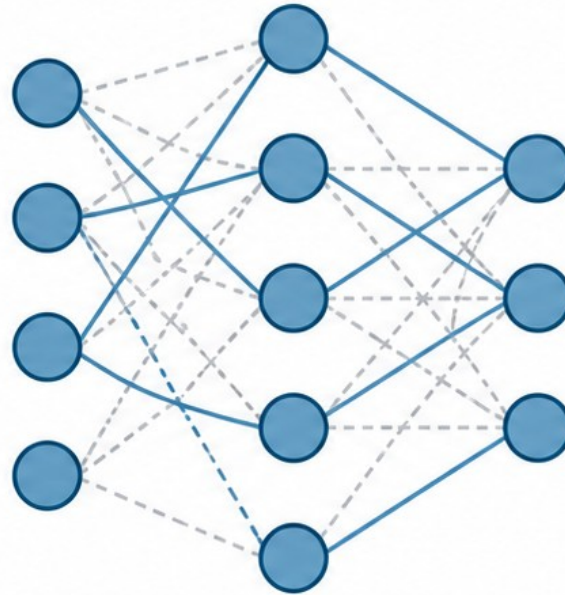
Quantization, pruning and distillation

Quantization



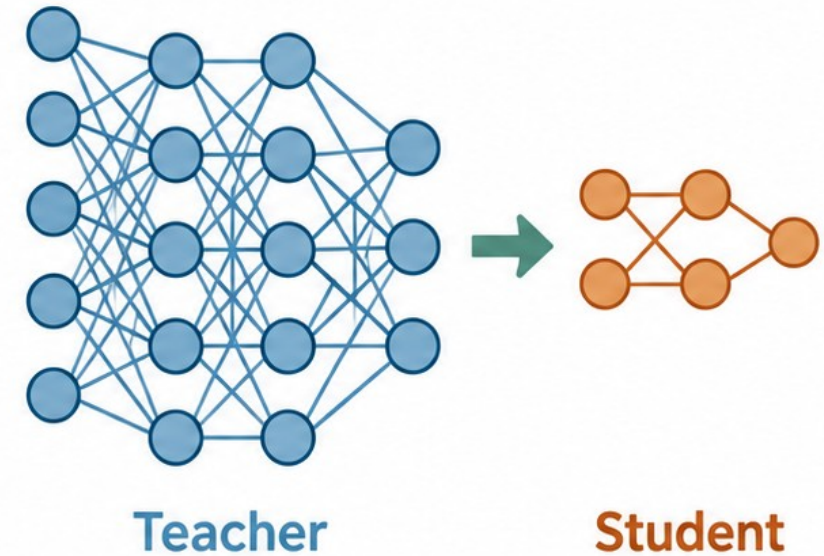
Fewer bits for values

Pruning



**Fewer active weights
or connections**

Distillation



**A smaller model learns
from a larger teacher**

Three questions to discuss

01

**The range doubles, but the number of levels stays fixed.
What happens to the step size?**

02

**With scale 0.25 and zero-point 0, which reconstructed
value represents 0.90 after rounding?**

03

**If the weights use half as many bytes, must inference
be twice as fast?**

The central ideas

Precision

A numeric format limits which values we can represent.

Mapping

Scale and zero-point connect integer codes to real values.

Behaviour

Approximation can change the model's predictions.

Next: comparing converted models and interpreting measured results.

Further reading

[LiteRT post-training quantization](#)

Overview of the quantization approaches available in LiteRT.

[LiteRT 8-bit specification](#)

The scale and zero-point mapping used for integer tensors.

[Post-training integer quantization](#)

The role of representative inputs and calibration.

[Quantization-aware training](#)

How training can account for quantization effects.